Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

1.1.1. **Sample the data and combine the files**

Due to the large size of each file and combined taxi datasets, direct processing of the complete data is time consuming and require significant resource to load entire data. To improve processing efficiency and reduce memory usage, a sampling strategy was applied by me.

A random sampling technique (5% of hourly data for each day) was used to extract a representative subset of records from each monthly dataset.

While fetching the record created date and hour column in dataset for easy computation

```
df["tpep_pickup_date"] = df["tpep_pickup_datetime"].dt.date

df["tpep_pickup_hour"] = df["tpep_pickup_datetime"].dt.hour
```

Sampling Code used:

```
df_sampled_hour = df_hour.sample(frac=0.05, random_state=42)
```

After sampling, all monthly datasets were combined into a single unified DataFrame using the concatenation method.

```
if sampled_monthData:

    sampled_yearData = pd.concat(sampled_monthData,
ignore_index=True)
```

After combining the data files into one DataFrame, converted the new DataFrame to a parquet file and store it to use directly.

```
sampled_yearData.shape

sampled_yearData.to_parquet('SampleData.parquet')
```

Outcome: After sampling and combining, a single cleaned and manageable dataset was created containing representative taxi trip records for the entire year. This consolidated dataset was then used for further cleaning, analysis, and visualization.

Combined data set contains 1896400 records, 22 columns which is easy to manage and to continue with further analysis in comparison to millions of records.

2. Data Cleaning

2.1. Fixing Columns

2.1.1. Fix the index

During the merging and sampling process, the original row indices became inconsistent and non-sequential. To resolve this issue, the index of the DataFrame was reset. Resetting the index ensured that each record had a unique and continuous identifier

```
df.reset_index(drop=True,inplace=True)

drop = True removes original index.
```

Removed store_and_fwd_flag flag as well. Since this variable does not Influence fare, distance, or passenger behavior, it can be considered irrelevant for this analysis.

```
df.drop('store_and_fwd_flag',axis=1,inplace=True)

inplace = True used to replace it in the existing df and no external assignment needed.
Axis = 1 to represent column wise.
```

Outcome: After resetting the index, the dataset had a clean and continuous indexing system. The removal of unnecessary columns ensured that only relevant variables were retained for further analysis.

2.1.2. Combine the two airport_fee columns

During initial analysis I found that both columns represented the same type of information but contained different proportions of missing values. The column Airport_fee contained 213,357 missing values, whereas airport_fee contained 1,747,917 missing values out of 1896400 records. This indicated that Airport_fee was completer and more reliable.

To resolve this inconsistency, the following steps were performed:

- First, records where both columns contained missing values were identified.

```
both_present = df[~df["airport_fee"].isna() & ~df["Airport_fee"].isna()]
```

No such records identified.

- Since Airport_fee had fewer missing values, it was selected as the primary column.
- Missing/null values in Airport_fee were filled using corresponding values from airport_fee.

```
df["Airport_fee"] = df["Airport_fee"].fillna(df["airport_fee"])
```

- After merging the information, the redundant airport_fee column was removed.

Outcome: After this process, a single consolidated Airport_fee column was retained with significantly fewer missing values.

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

After identifying and fixing negative values by replacing it with 0

```
numeric_cols = df.select_dtypes(include="number").columns

cols_with_negative_values = [
    col for col in numeric_cols if (df[col] < 0).any()
]

cols_with_negative_values
```

```
# fix these negative values

df.loc[df["extra"]< 0,"extra"] = 0
df.loc[df["mta_tax"]< 0,"mta_tax"] = 0
df.loc[df["improvement_surcharge"]< 0,"improvement_surcharge"] = 0
df.loc[df["congestion_surcharge"]< 0,"congestion_surcharge"] = 0
df.loc[df["Airport_fee"]< 0,"Airport_fee"] = 0


df["recalculate_total_amount"] = (
    df["fare_amount"]
    + df["extra"]
    + df["mta_tax"]
    + df["improvement_surcharge"]
    + df["congestion_surcharge"]
    + df["Airport_fee"]
    + df["tip_amount"]
    + df["tolls_amount"]
)

df["total_amount"] = df["recalculate_total_amount"]
df.drop(columns="recalculate_total_amount",axis=1, inplace=True)
```

The proportion of missing values was calculated for every variable by dividing the number of null values by the total number of records.

```
(df.isna().sum()/df.shape[0])*100
```

Outcome: This analysis identified columns with high, moderate, and low proportions of missing values, that can be handled accordingly

2.2.2. Handling missing values in passenger_count

Below analysis performed

1. Initially, the distribution of passenger counts was examined to identify the frequency of each value.

```
df["passenger_count"].value_counts().sort_index()
```

2. The analysis revealed that some records had a passenger_count value of 0, which is logically inconsistent for a completed taxi trip. Such entries were therefore considered invalid and treated as missing values. Hence replaced such values with nan.

```
df.loc[df['passenger_count']==0,'passenger_count'] = np.nan
```

3. The median was chosen as the imputation method because it is robust to outliers and provides a realistic estimate of typical passenger numbers. Furthermore, passenger count was found to have minimal influence on fare amount, trip duration, and other financial variables, making median imputation a suitable approach.

```
median_passangers_count = df["passenger_count"].median()

df["passenger_count"] = (
    df["passenger_count"]
    .fillna(median_passangers_count)
    .astype(int)
)
```

Outcome: After this process, all invalid and missing passenger count values were replaced with a representative median value.

2.2.3. Handle missing values in RatecodeID

To address this issue, all missing values in the RatecodeID column were filled with the value 99, which represents "Unknown or Null" according to the data dictionary.

```
df['RatecodeID'] = (df['RatecodeID'].fillna(99).astype(float))
```

Outcome: Following this transformation, all missing values in the RatecodeID column were replaced with a valid categorical code. In future analysis, this column converted to integer format to reflect its categorical nature more accurately.

2.2.4. Impute NaN in congestion_surcharge

During my analysis I found multiple null values in congrestion_surcharge column. Since a missing congestion surcharge typically indicates that no surcharge was applied to the trip, these null values were interpreted as zero

charges rather than unknown values. Therefore, I replaced all missing entries with 0.

```
df['congestion_surcharge'].isna().sum()
df['congestion_surcharge'] =
(df['congestion_surcharge'].fillna(0).astype(float))
```

Post the I ran below query to check other null value column wise:

```
df.isna().sum()
```

Replace nan with 0 for Airport_fee column.

```
df["Airport_fee"] = (df["Airport_fee"].fillna(0).astype(float))
```

Recalculated the total amount to remove null values from total_amount

Outcome: After applying this operation, the dataset contained only complete records with no missing values across any columns. This improved the reliability of subsequent statistical analysis and visualizations.

.

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

Below steps performed for outlier analysis:

1. For each column use **describe ()** to get mean, max, std, median, quantile and distribution of data.
   e.g.: **df['trip_distance'].describe().round(2)**

2. Below outliers handled.

   - Removed records where trip distance is more than 250 miles. It is highly unlikely for city cabs to travel more than 250 miles as a single trip.

     df = df[df["trip_distance"] <= 250]

- Removed the records where passenger count is more than 6

```
df = df[df['passenger_count'] <= 6]
```

- Removed the records where trip distance is nearly 0 and fare amount is more than 300.

```
df = df[~((df["trip_distance"] <= 0.1) &
(df["fare_amount"] > 300))]
```

- Removed the records where trip distance and fare amount zero but pickup and drop off zones are different.

```
df = df[~(
   (df["trip_distance"] == 0) &
   (df["fare_amount"] == 0) &
   (df["PULocationID"] != df["DOLocationID"])
)]
```

- Removed records where drop off time is before pickup time

```
df["trip_duration_in_sec"] = (
   df["tpep_dropoff_datetime"] -
df["tpep_pickup_datetime"]
).dt.total_seconds()

invalid_time_trips = df[df["trip_duration_in_sec"] < 0]
df = df[df["trip_duration_in_sec"] >= 0]
```

- Removed records where trip timing is more than 24 hrs but distance covered is very less.

```
df = df[((df["tpep_dropoff_datetime"] -
df["tpep_pickup_datetime"]
).dt.total_seconds()/3600)<=24]
```

- Removed records where trip duration is zero but distance covered is non zero

```
df = df[~((((df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"] ).dt.total_seconds() / 3600)==0) & df['trip_distance'] > 0) ]
```

- Entries where payment type is 0 replace it with 99(unknown/null)

```
df.loc[df['payment_type']==0,'payment_type'] = 99
```

- Removed one record with fare in lacs

```
df = df[df['fare_amount']<1500]
```

Outcome: Outliers were primarily examined in key numerical variables such as trip_distance, fare_amount, total_amount, trip_duration, and tip_amount, as these variables have a direct impact on revenue analysis and trip behavior insights. After identifying these cases, such records were removed from the dataset to maintain analytical reliability.

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

- VendorID: Categorical
- tpep_pickup_datetime: Datetime
- tpep_dropoff_datetime: Datetime
- passenger_count: Discrete Numerical
- trip_distance: Numerical Continuous
- RatecodeID: Categorical
- PULocationID: Categorical
- DOLocationID: Categorical
- payment_type: Categorical
- pickup_hour: Discrete Numerical (0,1,2,3,4....23)
- trip_duration: Numerical Continuous
- fare_amount:Numerical Continuous
- extra:Numerical Continuous
- mta_tax:Numerical Continuous

- tip_amount:Numerical Continuous
- tolls_amount:Numerical Continuous
- improvement_surcharge:Numerical Continuous
- total_amount:Numerical Continuous
- congestion_surcharge:Numerical Continuous
- airport_fee:Numerical Continuous

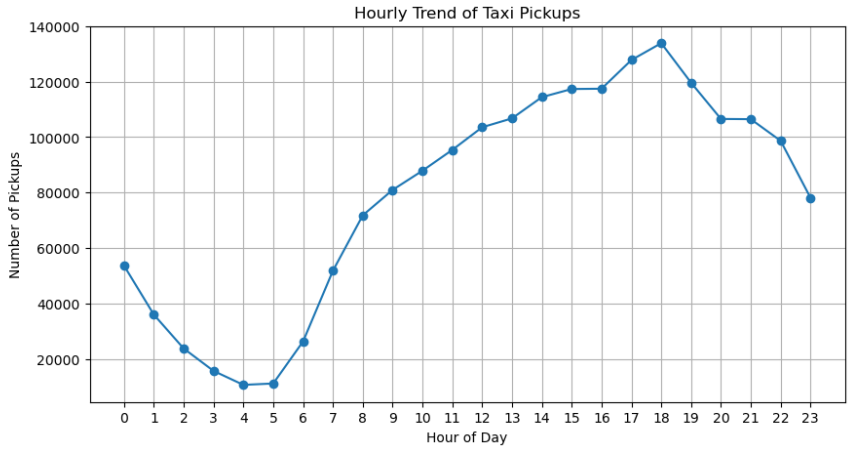
3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

Taxi pickups by hours

The dataset was grouped by pickup hour to analyze how taxi demand varies throughout the day. The number of trips initiated in each hour was calculated and visualized using a plot.

```
hourly_pickups = (
  df.groupby('tpep_pickup_hour')
    .size()
    .reset_index(name='pickup_count')
)
```

	tpep_pickup_hour	pickup_count
0	0	53641
1	1	35961
2	2	23744
3	3	15659
4	4	10626
5	5	11128
6	6	26289
7	7	51801
8	8	71780
9	9	80946
10	10	87871
11	11	95380
12	12	103543
13	13	106743
14	14	114413
15	15	117337
16	16	117443
17	17	127796
18	18	133876
19	19	119602
20	20	106538
21	21	106459
22	22	98653
23	23	78103



Interpretation:

- The hourly pickup distribution shows clear demand patterns across the day.
- Pickup activity is lowest during late-night and early-morning hours (approximately 3 AM to 5 AM).
- Demand increases steadily after morning hours.
- Peak pickup volumes are observed during evening peak hours around 6 pm.
- A gradual decline is visible after 6pm.

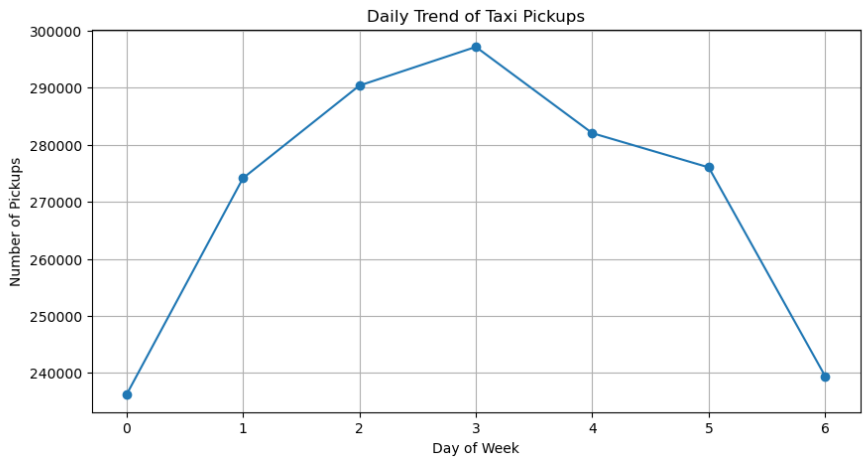
Taxi pickups by days of the week

To analyze weekly travel patterns, the pickup datetime was converted into weekday values and added as separate column in dataset, where Monday is represented as 0 and Sunday as 6.

```
df['weekday'] = df['tpep_pickup_datetime'].dt.weekday   # Monday - 0

daily_pickups = (
   df.groupby('weekday')
     .size()
     .reset_index(name='count')
)
```

	weekday	count
0	0	236195
1	1	274131
2	2	290369
3	3	297146
4	4	282030
5	5	276030
6	6	239431



Interpretation:

- Taxi usage is lowest on Mondays and Sundays, indicating relatively lower travel activity at the beginning and end of the week.
- Demand gradually increases from Tuesday to Thursday, reaching its peak on Thursday.
- Friday and Saturday pickup volumes not drastically less in comparison to weekdays, likely due to social outings, shopping, and weekend activities.
- Sunday experiences a noticeable decline in comparison to Saturday, reflecting reduced commuting.

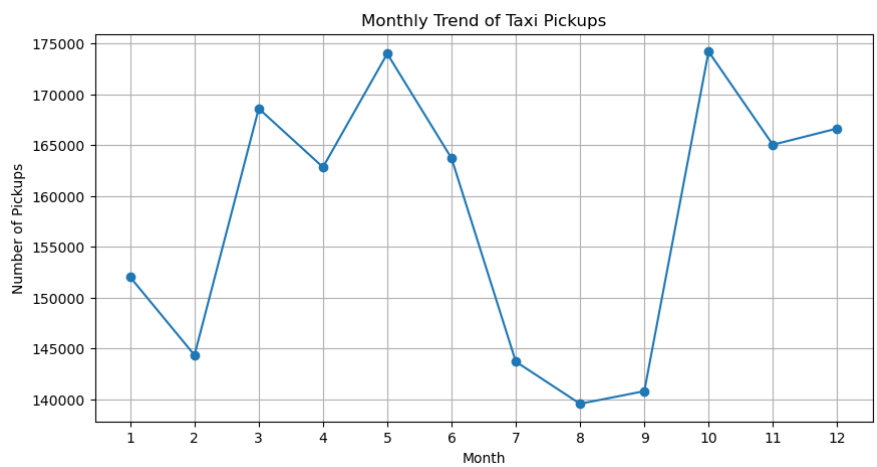
Taxi pickups by Month:

To identify seasonal patterns in taxi usage, taxi pickups were analyzed on a monthly basis. The pickup datetime column was used to extract the month, added as new column in dataset, and the total number of trips in each month was calculated.

```
df['month'] = df['tpep_pickup_datetime'].dt.month

monthly_pickups =
(df.groupby('month').size().reset_index(name='count'))
```

	month	count
0	1	152009
1	2	144349
2	3	168582
3	4	162819
4	5	173975
5	6	163697
6	7	143706
7	8	139562
8	9	140801
9	10	174185
10	11	165027
11	12	166620



Interpretation:

- Taxi demand is relatively lower during February.
- Pickups increase during March to May, reaching a peak in May.
- A decline is observed in (July to September), possibly due to vacations and reduced business activity.
- Another peak occurs in October, indicating increased travel after the summer period.
- Demand remains stable during November and December, likely influenced by holiday travel and shopping seasons.
- Overall, October and May recorded the highest pickup volumes, while August showed the lowest demand.

3.1.3. Filter out the zero/negative values in fares, distance and tips

Analyze using below query:

```
print(df[df['fare_amount']<=0]. shape[0])  -- 561
print(df[df['tip_amount']<=0].shape[0])   --  435231
print(df[df['total_amount']<=0].shape[0]) --331
print(df[df['trip_distance']<=0].shape[0]) --37619
```

A new dataset, df_non_zero, was created by applying the following criteria:

- Only records with positive fare_amount were retained.
- Trips with positive total_amount were included.
- Tip amounts were required to be non-negative.
- Trips with positive distance were selected.
- Trips with zero distance were retained only when the pickup and drop-off locations were the same, indicating possible short or stationary trips.

```
df_non_zero = df[
    (df["fare_amount"] > 0) &
    (df["total_amount"] > 0) &
    (df["tip_amount"] >= 0) &
    ((df["trip_distance"] > 0) | ((df["trip_distance"] == 0) &
(df['pu_location_id'] == df['do_location_id']))))].copy()
```

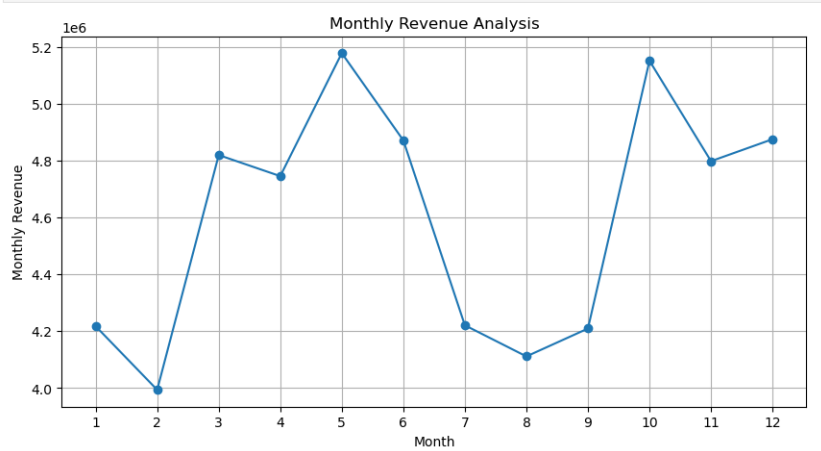
After applying these filtering conditions, a cleaned dataset containing only valid trip records was obtained. The shape of this refined dataset was verified using

```
df_non_zero.shape()  -- ( 1873026,22)
```

3.1.4. Analyse the monthly revenue trends

To analyze monthly revenue trends below steps were taken

1. A cleaned dataset (df_non_zero) containing valid trip records was used to ensure accurate revenue calculation.
2. We initially used pickup by date column and created month column in our dataset (Mentioned in above analysis).
3. The dataset was grouped by month and total_amount for each month was calculated using summing the total_amount column.
4. A line chart was created to visualize monthly revenue trends.
   X-axis: Month (1 to 12)
   Y-axis: Monthly revenue.



As from line chart it is clearly visible that:

1. Revenue was lowest in February.
2. Revenue was maximum during May and October.
3. Noticeable declined during July and August.
4. Revenue remains stable during November and December

Insights:

1. March to May showed inclined trend of revenue growth, likely due to increased travel and tourism.
2. End of year months shows steady revenue due to holiday travel and shopping.
3. July to September show lower revenue, possible due to vacation and reduced business travel.

3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

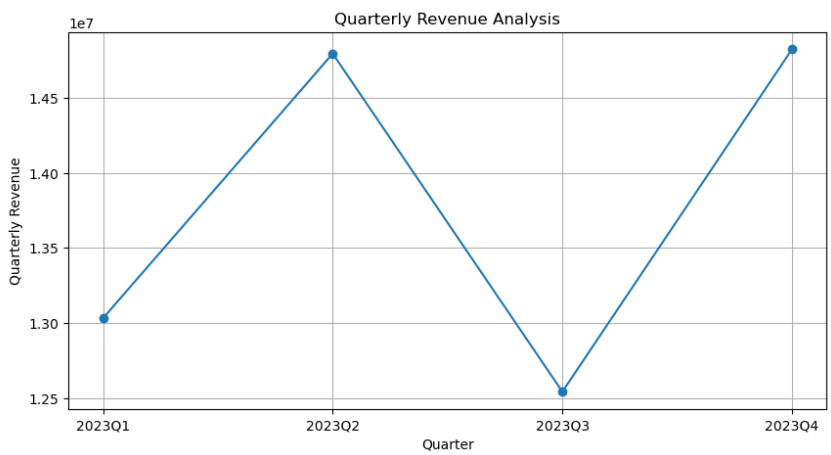
To determine the contribution of each quarter to annual revenue, at first using the pickup datetime column, separate column was created to show quarter.

df_non_zero["quarter"] = df_non_zero["tpep_pickup_datetime"].dt.to_period("Q")

Then the total revenue for each quarter was calculated by summing the total_amount column.

quarterly_summary = df_non_zero.groupby("quarter")["total_amount"].sum().reset_index(name="quarterly_revenue")

A line plot was used to visualize quarterly revenue trends, enabling easy comparison across quarters.



Insights:

- Revenue shows a strong increase from Q1 to Q2, reflecting higher travel demand during late spring and early summer.
- A noticeable decline is observed in Q3, indicating a seasonal slowdown, possibly due to vacation periods and reduced business travel.
- Revenue rises again in Q4, matching or even exceeding Q2 levels, likely driven by year-end holidays, festive activities, and increased urban travel.

3.1.6. Analyse and visualise the relationship between distance and fare amount

- The dataset was filtered to include only valid trips with non-zero fare and total amounts (df_non_zero) to ensure data reliability.

  non_zero_trip_duration = df_non_zero[df_non_zero["trip_distance"] > 0]

- Pearson correlation analysis was applied to measure the linear relationship between trip_distance and fare_amount.

  corr_nzdata = non_zero_trip_duration[["trip_distance", "fare_amount"]].corr(method="pearson")

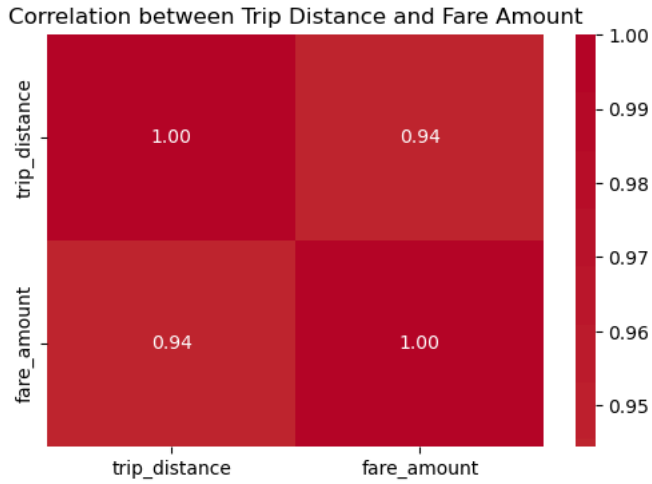
- Correlation was initially calculated using all records, including trips with zero distance to get a clearer understanding that how outliers can impact correlation.

  corr_data = df_non_zero[["trip_distance", "fare_amount"]].corr(method="pearson")

- Trips with trip_distance = 0 were then removed, as such records may represent data entry errors, short stationary trips, or system anomalies.

  non_zero_trip_duration = df_non_zero[df_non_zero["trip_distance"]> 0]

- The correlation was recalculated after removing zero-distance trips to obtain a more accurate estimate.
- A heatmap visualization was created using Seaborn to clearly represent the strength and direction of the correlation.
- The darker shed of colour represents stronger relationship between trip distance and fare amount.
- When all records were included, the correlation coefficient between trip distance and fare amount was approximately **0.92**, indicating a strong positive relationship.
- After removing trips with zero distance, the correlation increased to approximately **0.94,** indicating a stronger relationship.
- This improvement suggests that zero-distance trips slightly weakened the original correlation.



Insights:

The stronger correlation indicates that fare amount increases almost proportionally with trip distance

The presence of zero distance trip causes noise into the dataset and may lead to inaccurate analysis.

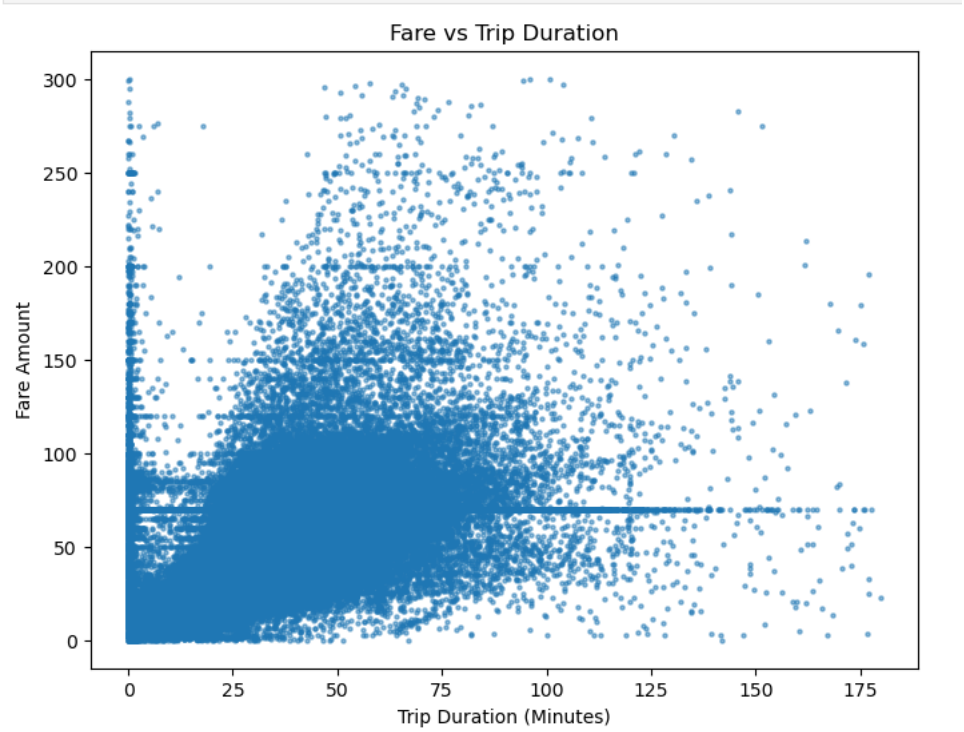
3.1.7. Analyse the relationship between fare/tips and trips/passengers

<u>Fare Amount and Trip Duration:</u>

1. Trip duration was first calculated in mins using the difference between drop off and pickup date time.
2. Outliers were filtered from dataset. Trips with a duration greater than 180 mins and fares exceeding 300 were removed.

```
df_clean = df_non_zero[
    (df_non_zero["trip_duration_min"] < 180) &   # < 3 hours
    (df_non_zero["fare_amount"] < 300)
]
```

3. A scatter plot was then generated to visualise the relationship between trip duration and fare amount. Transparency (alpha) was applied to improve readability due to the high volume of data points. Each point represents a single taxi trip, with trip duration on the x-axis and fare amount on the y-axis.



Insights:

The plot shows that fares tend to increase as trip duration increases. Most trips are concentrated between 10 and 80 minutes, with fare amounts typically ranging between 20 and 120.

Correlation analysis was performed before and after filtering:

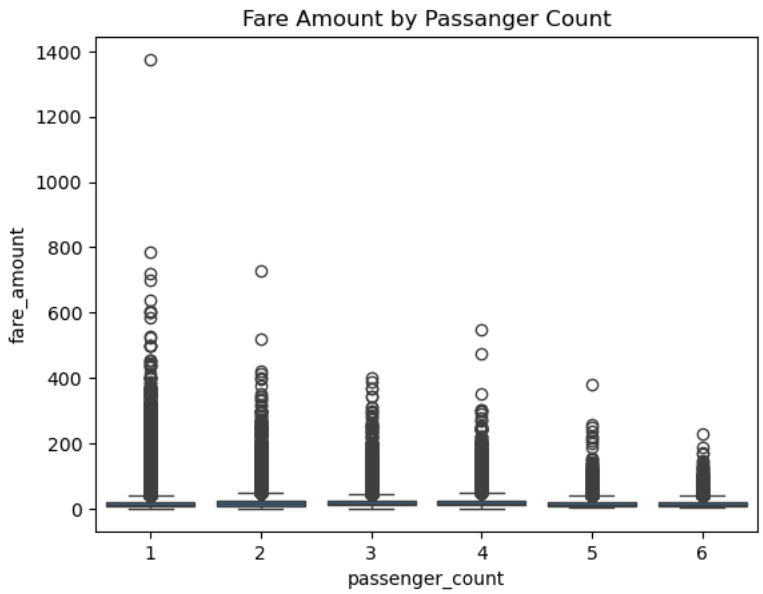
- The unfiltered dataset showed a weaker correlation due to the presence of outliers. (0.277)
- The filtered dataset showed a stronger and more consistent positive correlation. This confirms that trip duration is an important factor influencing fare amount. (0.824)
- With this analysis we concluded that trip duration is also a key driver of fare pricing in taxi services.

<u>Fare Amount and Number of passengers:</u>

1. First, the Pearson correlation coefficient was calculated between passenger_count and fare_amount to quantify the strength and direction of their relationship.

   corr = df_non_zero["passenger_count"].corr(df_non_zero["fare_amount"])

2. Next, a box plot was generated using Seaborn, with passenger count on the x-axis and fare amount on the y-axis. The box plot was selected as it effectively displays the distribution, median, interquartile range, and presence of outliers across different passenger groups.



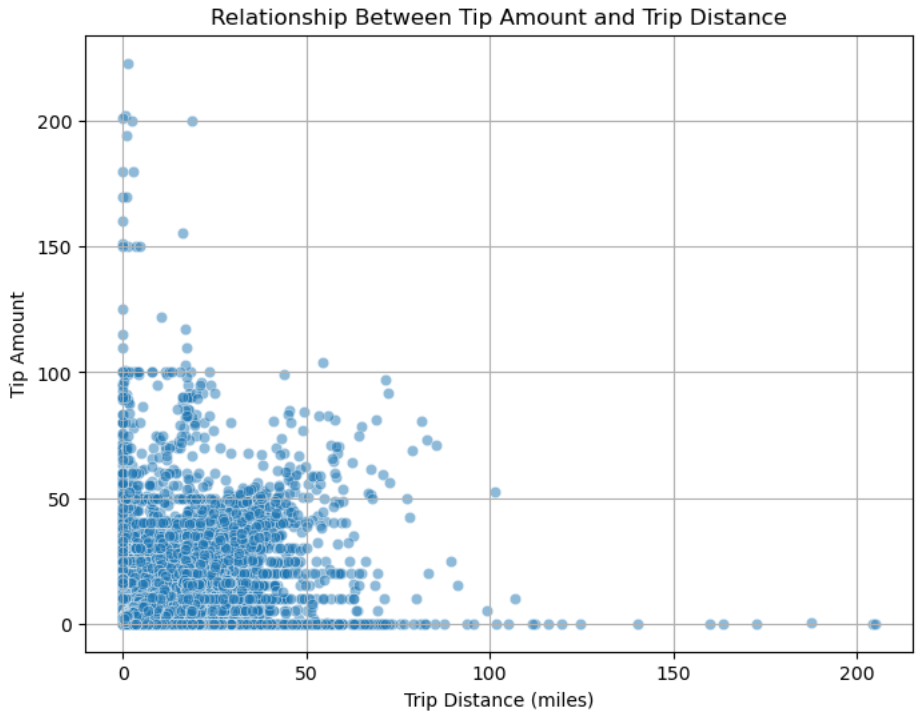
Insights:

- Across all passenger categories, the median fare remains relatively consistent, with only slight variations. (Displayed via horizontal line)
- Most fares are concentrated in a similar range regardless of the number of passengers.
- My data set contains trips mostly with passenger count 1 (1449706) or 2(276488). Hence large number of outliers also present in this category.
- Correlation is 0.042 which indicate a negligible linear relationship between passenger count and fare amount. It means passenger count have very little or no impact on fare amount.
- The high frequency of single-passenger trips suggests that most rides are taken individually.

<u>Tip and Trip Distance:</u>

1. First, the Pearson correlation coefficient was calculated between tip_amount and trip_distance to quantify the strength and direction of their linear relationship.
   corr = df_non_zero["tip_amount"].corr(df_non_zero["trip_distance"])

2. Next, a scatter plot was created using Seaborn, with trip distance on the x-axis and tip amount on the y-axis.



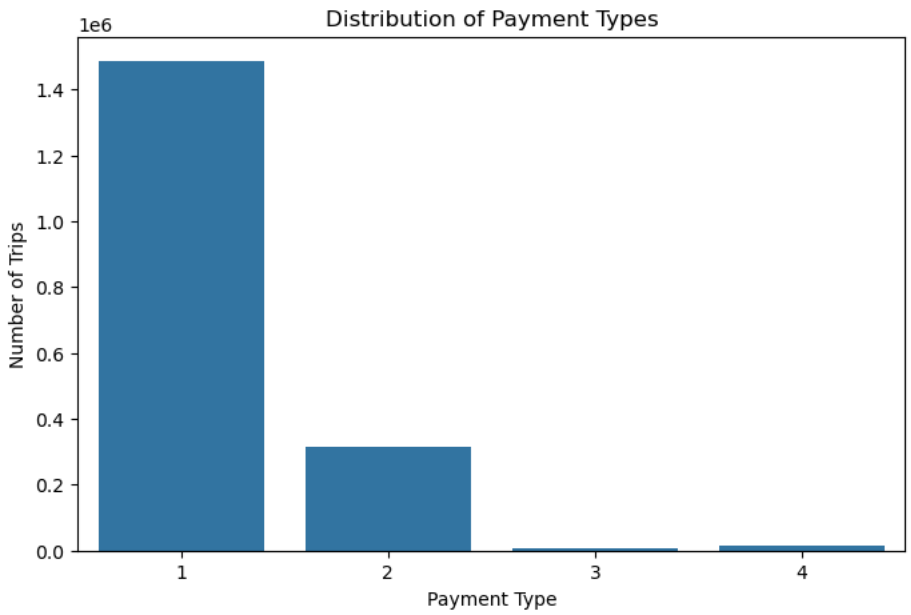
Insights:

- Most trips are concentrated within the range of 0 to 50 miles, where the majority of tipping activity occurs. Within this range, tip amounts typically fall between 0 and 60, with some higher outliers.
- Several observations with very high tip values (above 150) are visible, especially for short and medium-distance trips. These represent exceptional tipping behavior, possibly influenced by service quality, passenger satisfaction.
- Passengers on usually long trips may be less inclined to provide tips due to high fare or maybe they thought that after such fare a fixed sufficiently low tip is enough.
- Correlation is 0.575 indicating a moderate relationship between tip amount and trip distance.

- Tipping behavior is not strictly proportional to distance, as significant variation exists at all distance levels.

3.1.8. Analyse the distribution of different payment types

1. Unknown Payment Type code (99) removed to ensure analysis is focus on valid payment method.
2. A bar chart (count plot) was created using Seaborn to display the frequency distribution of trips across different payment types.
3. Later relative proportions of each payment type were calculated using below query to find out percentage share
df_clean["payment_type"].value_counts(normalize=True)*100



Insights:

- Payment type 1 which is credit card is used by more than 80% of the trips. It indicates that credit card is preferred due to widespread availability, due to convenience or default system setting.
- Payment type 2 which is cash is used by approximately 17 percent of users.
- No Charge and dispute categories are very less indicating strong system for payments and its effectiveness.
- From a business perspective, this skewed distribution highlights the importance of maintaining robust support for Payment Type 1

while investigating the reasons behind the minority preference for Payment Type 2.

3.1.9. Load the taxi zones shapefile and display it

- The shapefile taxi_zones.shp was read using the geopandas library, which allows integration of spatial and tabular data.
- The dataset's dimensions were checked using. head () and. shape to understand its structure and the number of records.
- Data Cleaning: Duplicate records were identified and removed based on the LocationID field. This ensures each geographic zone is uniquely represented. The dataset was then reindexed for consistency.

  zones = zones.drop_duplicates(subset="LocationID").reset_index()

Conclusion: The methodology applied—importing, exploring, cleaning, and validating the shapefile—ensures a robust foundation for analysis of taxi trips.

3.1.10. Merge the zone data with trips data

- The initial trip dataset df was cleaned to remove invalid entries and df_non_zero used for further analysis.
- A join was performed using pd.merge, linking trips (pu_location_id) with geographic zones (LocationID). The inner join ensured only valid matches were retained.

  merge_data = pd.merge(df_non_zero,zones,left_on="pu_location_id",right_on="LocationID",how="inner")

- Validation: The merged dataset (merge_data) was inspected to confirm alignment between trip records and zone boundaries. This step guarantees that each trip is associated with a valid pickup zone.
  The initial dataset shows 1873026 records and merged dataset shows 1854310 records. Which indicate that there are around 18000 plus records with invalid pickup location id.

Conclusion: The merge successfully integrated trip records with geographic identifiers, creating a dataset that links operational data (trips) with spatial context (zones).

3.1.11. Find the number of trips for each zone/location ID

- The shape of df_non_zero and merge_data was checked to ensure consistency after merging.
- The merged dataset was grouped by LocationID to count the number of trips per zone.
  pickup_trips = merge_data.groupby("LocationID").size().reset_index(name="trip_count")

- Zones were sorted in descending order of trip counts, and the top 10 pickup zones were extracted for closer analysis.

	LocationID	trip_count
124	132	96601
228	237	88156
153	161	86901
227	236	79165
154	162	66333
130	138	64305
177	186	63901
221	230	62105
134	142	62048
162	170	55405

Conclusion:

The top 10 pickup zones accounted for a large share of trips, indicating strong clustering of demand.

Peripheral or less central zones showed significantly fewer pickups, reinforcing the uneven distribution of taxi usage across the city.

3.1.12. Add the number of trips for each zone to the zones dataframe

- Data Integration: The pickup_trips dataset, containing aggregated trip counts per LocationID, was merged with the cleaned and filtered zones dataframe to create zones_with_trips dataframe.
- Join Operation: A left join ensured that all geographic zones were retained, even if some had no recorded trips. This prevents loss of information and maintains full geographic coverage.
- Validation: The merged dataset (zones_with_trips) was checked to confirm that trip counts aligned correctly with their respective zones.

3.1.13. Plot a map of the zones showing number of trips

- Using Matplotlib and GeoPandas, a map was created where zones were shaded based on the number of pickup trips.
- Taxi operators can use this map to strategically allocate vehicles to high-demand zones, reducing wait times and maximizing efficiency.
- City planners can identify underserved areas and investigate whether low demand is due to infrastructure gaps, socioeconomic factors, or alternative transport options.
- Further zones with trip sorted based on trip count to get more information regarding busiest zones.

3.1.14. Conclude with results

To maximize efficiency, operators should dynamically allocate fleets to high-demand zones, maintain support for secondary payment methods, to remove high dependency on credit cards. Urban planners can leverage these insights to improve accessibility in underserved areas, integrate taxi services with public transit, and address mobility equity. Continuous monitoring and predictive modeling will further enhance responsiveness to shifting demand patterns.

Maintain support for payment type cash while prioritizing infrastructure for payment type 1 (Credit cards) to decrease heavy dependency on credit card system.

3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

Methodology:

- First of all, I added two new columns “pu_zone” and “do_zone” in df_non_zero dataset.
  zone_map = zones.set_index("LocationID")["zone"]

  df_non_zero["pu_zone"] =
  df_non_zero["pu_location_id"].map(zone_map)
  df_non_zero["do_zone"] =
  df_non_zero["do_location_id"].map(zone_map)

- Remove records with unknown pickup or drop-off zones

  df_non_zero = df_non_zero[
  df_non_zero["pu_zone"].notnull() &
  df_non_zero["do_zone"].notnull()].copy()

- Filter out records with zero trip distance and zero time taken. and calculated trip duration in hours.

  ```
  df_non_zero = df_non_zero[
     (df_non_zero["trip_duration_min"] > 0) &
     (df_non_zero["trip_distance"] > 0)
  ]

  df_non_zero["trip_duration_hr"] =
  df_non_zero["trip_duration_min"] / 60
  ```

- Created new route column to get pick off and drop zone combined information.
  ```
  df_non_zero["route"] = (
     df_non_zero["pu_zone"].astype(str)
     + " -- " +
     df_non_zero["do_zone"].astype(str)
  )
  ```
- Group by Route and pickup hour and calculate average trip distance, average trip duration and total trip count.

- Once we have all this data, we also calculated average speed per route in every hour.

```
route_hour_stats["speed_mph"] = (
    route_hour_stats["avg_distance"] /
    route_hour_stats["avg_duration"]
)
```

- To make our analysis more refined we avoid routes with a smaller number of trips. Routes with trips less than 20.
- Routes were sorted by speed, and the slowest 10 routes per hour were identified.

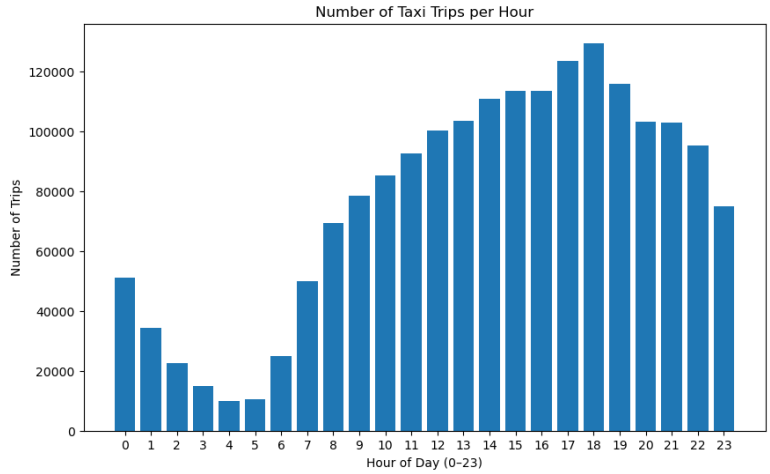
Analysis and Insights:

- The methodology successfully highlights slowest routes by time of day, accounting for both traffic congestion and demand patterns.
- Certain routes consistently appear among the slowest, often connecting busy commercial or transit hubs.
- Filtering out low-trip routes ensures that results are not skewed by outliers or rare trips.
- Slow speeds during specific hours confirm the impact of rush-hour traffic on taxi operations.
- Identifying slow routes helps operators anticipate delays, adjust pricing, or reallocate fleets.
- Persistent slow zones may indicate areas needing infrastructure improvements or traffic management interventions.

3.2.2. Calculate the hourly number of trips and identify the busy hours

Methodology:

- Trips were grouped by tpep_pickup_hour to calculate the number of trips per hour.
- The hourly trip counts were sorted to identify the busiest hour.
- A bar chart was plotted to display the number of trips across all 24 hours.



Analysis and Insights:

- The busiest hour was 18:00, with 129,434 trips, reflecting peak demand during evening commute hours. The evening peak aligns with end-of-workday travel, highlighting taxis as a preferred mode of transport during rush hour. Taxi operators should anticipate higher demand around 6 PM and ensure adequate fleet availability.
- Late-night and early-morning hours (0–6 AM) show significantly fewer trips, consistent with lower passenger activity.

3.2.3. Scale up the number of trips from above to find the actual number of trips

Methodology:

- The dataset was based on a 5% hourly sample (sample_fraction = 0.05). To estimate the actual number of trips, the observed counts were scaled up by dividing by the sampling fraction.
- Trips were grouped by pickup hour (tpep_pickup_hour) and sorted to identify the busiest hours.
- The top five busiest hours were extracted.

Insight:

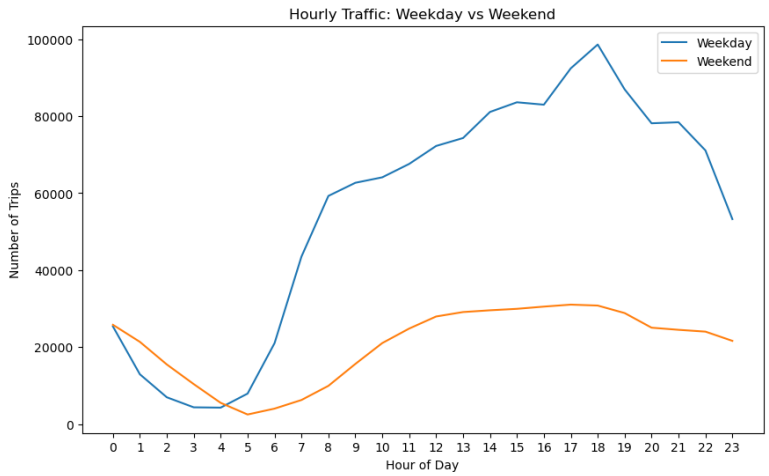
18	2588680.0
17	2469820.0
19	2316560.0
15	2271540.0
16	2270620.0

- The busiest hours were concentrated in the late afternoon and evening.
- Taxi demand is not evenly distributed across a day but peaks sharply in the evening.
- Fleet allocation should prioritize late afternoon and evening hours to reduce wait times and maximize efficiency.
- Operators can apply surge pricing or targeted promotions to balance supply and demand.

3.2.4. Compare hourly traffic on weekdays and weekends

Methodology:

- Created day_type column based on Weekday column value. Wherever weekday column value is less than 5 its treated as weekday and weekday column value with 5 and 6 is treated as weekend for day_type column value.
- Trips were then further categorized into Weekday (Monday–Friday) and Weekend (Saturday–Sunday) using the weekday column.
- For each category, trips were grouped by pickup hour (tpep_pickup_hour) to calculate hourly trip counts.
- Line plots were created to compare weekday and weekend traffic patterns across the 24-hour cycle.
- Counts were checked to ensure correct classification and representation of both weekday and weekend trips.



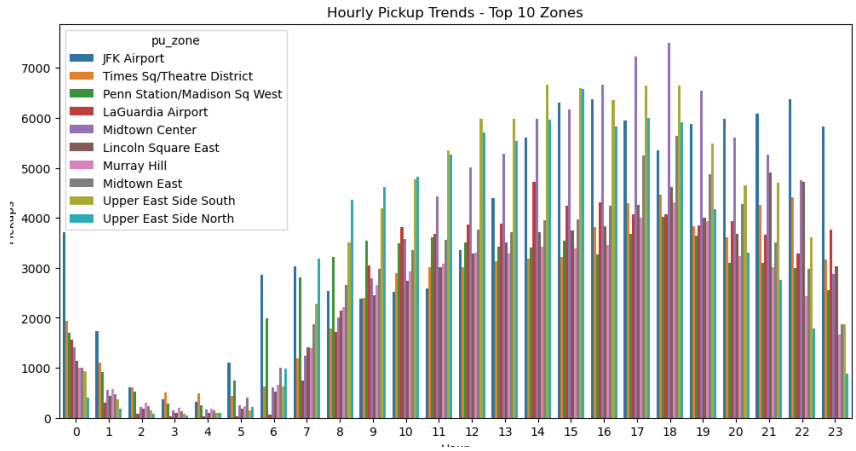
Analysis and Insights:

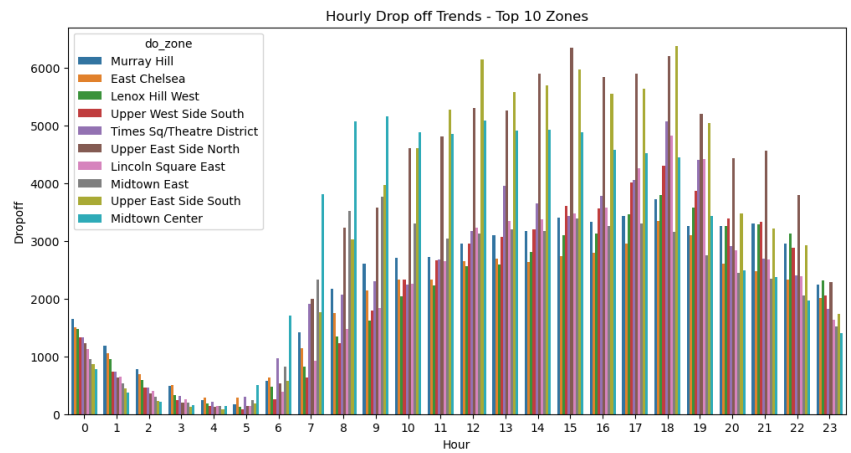
- Early morning cab demands are higher on weekend than weekdays.
- Evening demand remains steadier, reflecting leisure and social travel rather than commuter traffic.
- Weekday traffic patterns are strongly shaped by work schedules, with peaks during evening commute hours.
- On Weekend Increase fleet presence during late-night and early-morning hours (12–5 AM) to accommodate nightlife and leisure travel.
- Operators can assign a smaller but sufficient pool of drivers to weekends, focusing on nightlife and entertainment districts.

3.2.5. Identify the top 10 zones with high hourly pickups and drops

Methodology:

- Trips were grouped by Zone and tpep_pickup_hour for pickups and tpep_dropoff_hour for drop off initially.
- Instead of total counts, the mean hourly pickups and drop-offs were calculated for each zone. This highlights zones with consistently high activity across the day rather than just those with large totals.
- Zones were sorted by their average hourly pickups and drop-offs, and the top 10 zones were identified for each.
- Bar plots were created to show hourly pickup and drop-off trends in these top zones





Analysis and Insights:

- Airports (JFK, LaGuardia) and central business districts (Times Square, Midtown, Penn Station) dominate the top 10 pickup zones. These areas show strong commuter and traveler demand, which started increasing from morning hours and its peaks during evening hours.
- Allocate fleets to pick up hotspots (airports, transit hubs) during peak commute hours.
- Maintain balanced coverage in zones like Time square and Midtown, which show both strong pickups and drop-offs.
- Surge pricing can be applied in pickup hotspots during peak hours.

3.2.6. Find the ratio of pickups and dropoffs in each zone

Methodology:

- First, we group trips by pu_zone and count the number of trips.

- Then group by do_zone to get total drop-offs per zone.
- Merge the two data frame to align pickup and drop off counts.
- Use combine first approach to combine pu_zone column and do_zone column on first not nan value basis.
  zone_counts["zone"] = zone_counts["pu_zone"].combine_first(zone_counts["do_zone"])
- Fill missing value with zero and then later for records where drop off count is zero, replaced it with pd.NA to avoid infinite values in ratio.
  zone_counts = zone_counts[["zone", "pickups", "dropoffs"]].fillna(0)
  # To avoid divison by 0 -> Infinity we replace 0 with NA
  zone_counts["dropoffs"] = zone_counts["dropoffs"].replace(0, pd.NA)
- Computed the ratio pickup/dropoffs for each zone.

Analysis and Insights:

- High pickup dropoff ratio indicates that these zones are trip sources. They generate higher pickups than dropoffs.
- Lower pickup dropoff ratio indicates zones with high drop off than pickup.
- Ratio with 1 indicates zones requiring continuous fleet coverage as pickup and drop offs are same.
- Deploy more taxis to high-ratio zones during peak hours to capture demand.
- Apply surge pricing in pickup-heavy zones during demand spikes.

3.2.7. Identify the top zones with high traffic during night hours

Methodology:

- Pickup and drop off data filtered based on tpep_pickup_hour and tpep_dropoff_hour respectively to get trips in night hours (11 PM – 5 AM).

```
night_pickups_df = df_non_zero[

   (df_non_zero["tpep_pickup_hour"] >= 23) |
(df_non_zero["tpep_pickup_hour"] <= 5)

]
```

```
night_dropoffs_df = df_non_zero[
    (df_non_zero["tpep_dropoff_hour"] >= 23) |
(df_non_zero["tpep_dropoff_hour"] <= 5)
]
```

- Filtered data grouped by pu_zone and do_zone.
- Zones were ranked by total counts during night hours, and the top 10 zones for pickups and drop-offs were identified.

Analysis and Insights:

- Airport and station dominate late night pickups
- Residential neighborhood shows high dropoffs which indicate passengers returning home after late night parties.
- Allocate fleets to nightlife and airport zones during late-night hours to capture demand.
- Offer promotions in drop-off-heavy residential zones to encourage pickups and balance flows.

3.2.8. Find the revenue share for nighttime and daytime hours

Methodology:

- Trips were categorized into night time (11PM-5AM) and remaining as day time and time_period column created in dataset.
- For each trip, the total amount was aggregated separately for daytime and nighttime trips based on time_period column.
- The revenue share was calculated separately for day and night time using below formula.

```
total = revenue_split["total_revenue"].sum()

revenue_split["revenue_percent"] = (
    revenue_split["total_revenue"] / total * 100
)
```

```
Day      19.328114
Night    19.783364
```

Analysis and Insights:

- Majority of trips occur during daytime hours, driven by commuting or business travel.

- Daytime revenue typically accounts for the largest share of total earnings. (almost 88 percent)
- Though fewer trips occur at night, average fares tend to be higher due to longer distances

```
df_non_zero.groupby("time_period")["trip_distance"].mean()
```

```
time_period
Day      3.375561
Night    3.961798
```

```
df_non_zero.groupby("time_period")["fare_amount"].mean()
```

```
time_period
Day      19.328114
Night    19.783364
```

- Maintain strong fleet presence during daytime to capture high-volume commuter demand.

- Ensure targeted coverage at night in airports, transit hubs, and entertainment districts to capture high-value trips.

3.2.9. For the different passenger counts, find the average fare per mile per passenger

Methodology:

- Trips with valid passenger count and trip distance were selected to avoid division error.
  valid_df = df_non_zero[
  (df_non_zero["trip_distance"] > 0) &
  (df_non_zero["fare_amount"] > 0) &
  (df_non_zero["passenger_count"] > 0)
  ]
- For each trip fare per mile calculated
  valid_df["fare_per_mile"] =
  (valid_df["fare_amount"]/valid_df["trip_distance"])
- For very short trips with high fare may impact our analysis hence taking upper cap of 0.99 percentile to remove extreme cases.
  valid_df =
  valid_df[valid_df["fare_per_mile"]<=valid_df["fare_per_mile"].
  quantile(0.99)]
- Trips were grouped by passenger_count. For each group, the average fare per mile was calculated.

- The average fare per mile was divided by the passenger count to obtain, fare per mile per passenger.

	passenger_count	avg_fare_per_mile_pg	fare_per_mile_per_passenger
0	1	7.660854	7.660854
1	2	7.488905	3.744453
2	3	7.607484	2.535828
3	4	7.555669	1.888917
4	5	7.537638	1.507528
5	6	7.619932	1.269989

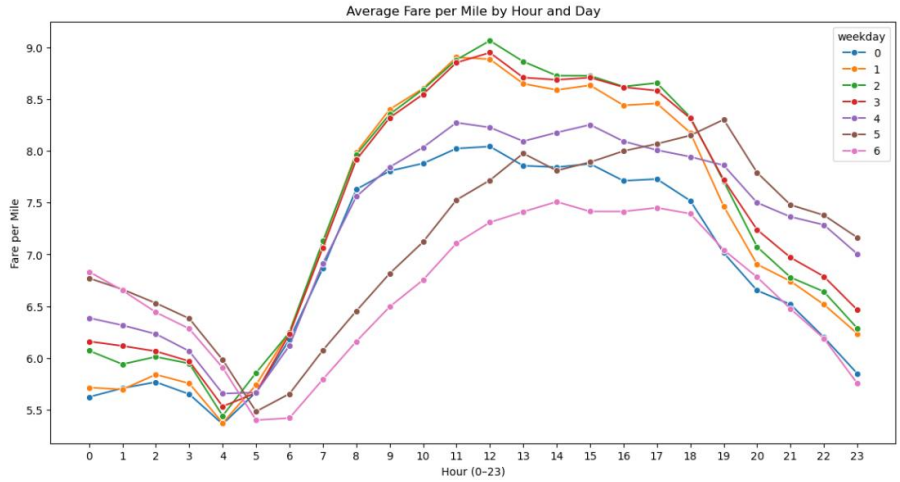
Analysis and Insights:

- Solo trips are the highest per passenger cost per mile, since the fare is not shared.
- Average fare per mile remains similar to solo trips, but per-passenger cost drops significantly. Indicating cost efficiency when sharing ride.
- Pooling service reduced the per passenger cost substantially.
- Encourage pooling services to maximize affordability and reduce congestion during peak time.

3.2.10. Find the average fare per mile by hours of the day and by days of the week

Methodology:

- We have extracted weekday and pickup hour and fare per mile initially for our analysis.
- Trips were grouped by weekday and tpep_pickup_hour to calculate mean fare per mile for each group.
- Line plot is used to show variation across hours and days.



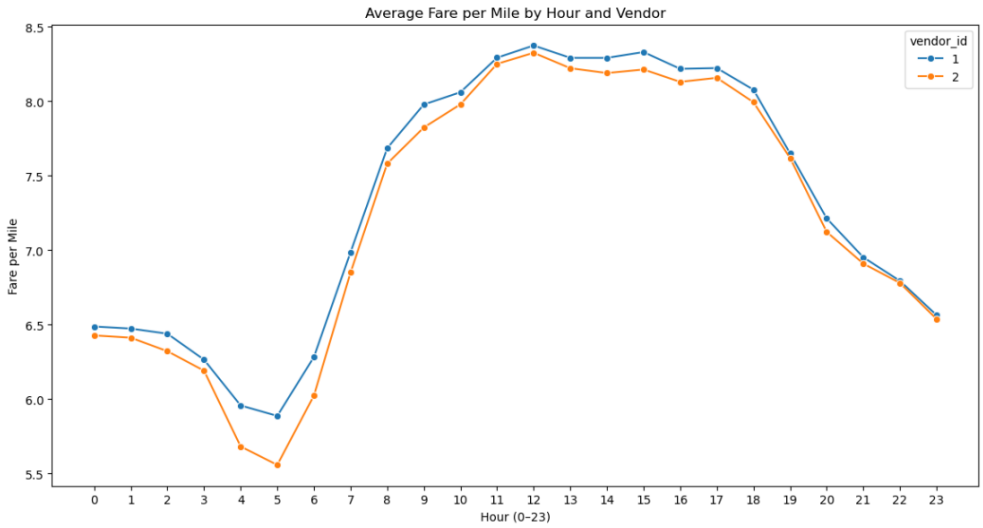
Analysis and Insights:

- Fare per mile tends to be lowest in the early morning hours (around 4–5 AM), when trip demand is low.
- On weekdays peak occurred during (11 am to 1 PM)
- Saturday indicates peak during (7PM - 9 PM) due to heavy demand.
- Weekdays (Monday–Friday) generally show higher fare per mile during office hours, reflecting work-related travel.
- Weekends (Saturday–Sunday) show flatter patterns during the day but higher late-night fares per mile, reflecting leisure and nightlife travel.
- Fleet allocation should prioritize weekday office hours to capture demand.
- Surge pricing can be applied during weekday peaks and weekend evening hours to maximize revenue.

3.2.11. Analyse the average fare per mile for the different vendors

Methodology:

- Trips were grouped by both vendor_id and tpep_pickup_hour.
- Within each group, the mean fare per mile was calculated.
- Pivot was created to get proper average fare per mile across hours difference between different vendors.
- Line plots were used to compare vendors across hours of the day



Analysis and Insights:

- Both vendors show relatively lower average fare per mile, dropping to their lowest around 4–5 AM (Vendor 1 approx. 5.95, Vendor 2 approx. 5.68). This reflects low demand and shorter trips during early morning hours.
- Fare per mile started increasing in morning around 9-10 and stay high during office hours.
- Both vendors follow the same pattern, with peaks during commute hours.
- Vendor 1 consistently records marginally higher fare per mile values than Vendor 2 across most hours.
- Vendor 2 shows smoother transitions, with slightly lower fare per mile values, possibly reflecting steadier pricing strategies.

3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

Methodology:

- Using Categorize_distance function created separate column distance_tier based on trip distance.

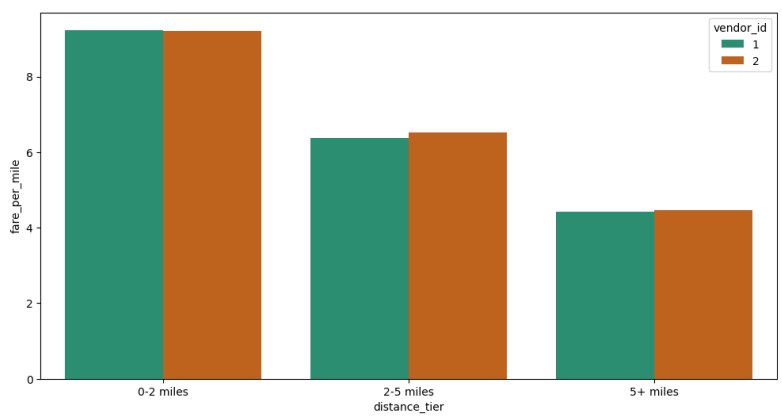
```
def categorize_distance(dist):
    if dist <= 2:
        return "0-2 miles"
    elif dist <= 5:
        return "2-5 miles"
    else:
        return "5+ miles"

valid_df["distance_tier"] =
valid_df["trip_distance"].apply(categorize_distance)
```

- Average fare per mile was computed for each vendor within each tier

```
tier_summary = (
    valid_df
    .groupby(["vendor_id", "distance_tier"])["fare_per_mile"]
    .mean()
    .reset_index()
)
```

- Bar plots were used to highlight differences across vendors in tiered fashion.



Analysis and Insights:

- Both vendors tend to have higher fare per mile values for short trips.
- Fare per mile for 2-5 miles trips decreases compared to short trips.
- Vendor 2 charge slightly lower for short trips but higher fare per miles in comparison to vendor 1 for medium and long trips as well.
- Vendors should emphasize efficiency and availability in short trips, where per-mile costs are highest.

- Long trips are revenue-rich but per-mile cheaper, vendors can differentiate by offering better long-distance rates in comparison to other vendors to get advantage.

3.2.13. Analyse the tip percentages

Tip percentage based on Trip Distance:

Methodology:

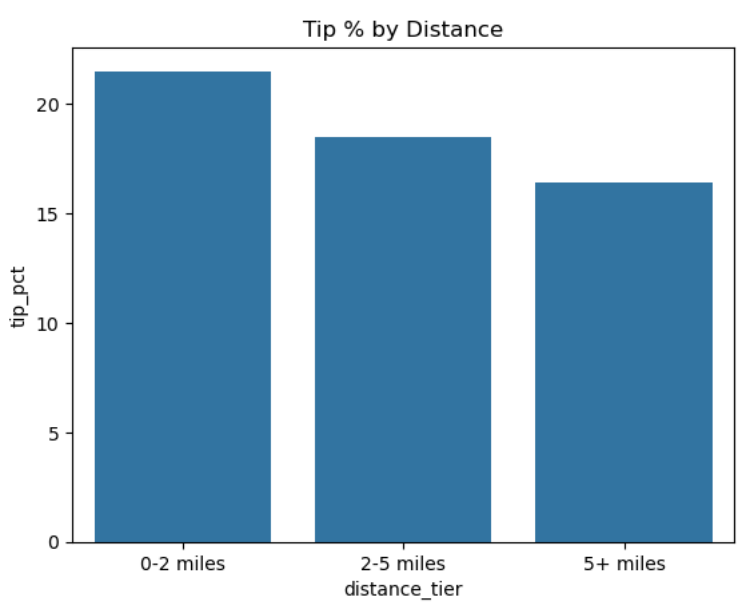
- Initially we have categorized trips into three tiers (0–2 miles, 2–5 miles, 5+ miles) and created a new column (distance_tier).
- Tip percentage was calculated using below formula

  df_non_zero["tip_pct"] = df_non_zero["tip_amount"] / df_non_zero["fare_amount"] * 100

- Remove extreme tip percentage cases for our analysis and created a new dataset by removing outliers.

  upper = df_clean["tip_pct"].quantile(0.99)
  df_clean = df_clean[df_clean["tip_pct"] <= upper]

- Average tip percentage was computed for each tier.
- Bar plots were used to highlight differences across distance categories.



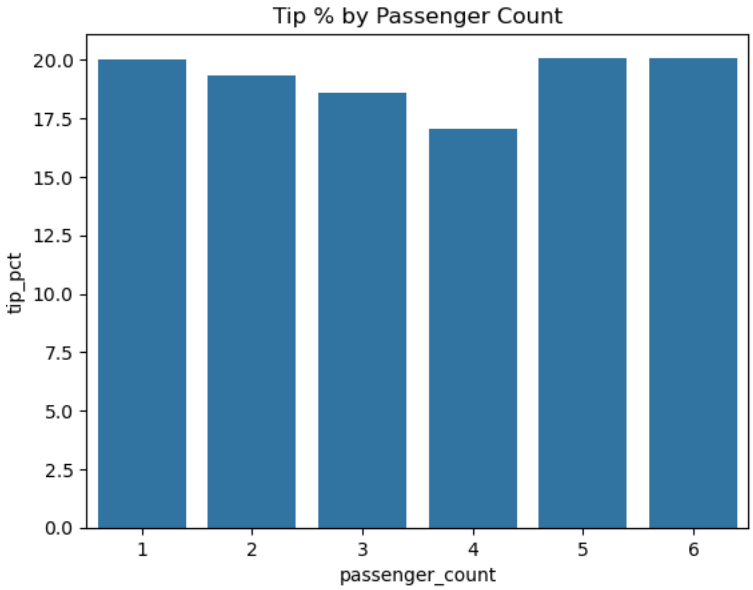
Analysis and Insights:

- Tip percentage is higher for short trips, as it may see as quick and easy, so riders may tip generously relative to fare
- On long trips, riders may feel a fixed tip of lower amount is sufficient as the fare amount is already a lot for long trips.
- Some passengers (mostly travelers) consistently tip low, influenced by local tipping practice.

Tip percentage based on Passenger count:

Methodology:

- Tip percentage was calculated relative to fare amount and then remove extreme tip percentage cases for our analysis.
- Trips were grouped by passenger_count.
- Average tip percentage was computed for each passenger count group.
- Visualization: Bar plots were used to highlight differences across passenger counts.



Analysis and Insights:

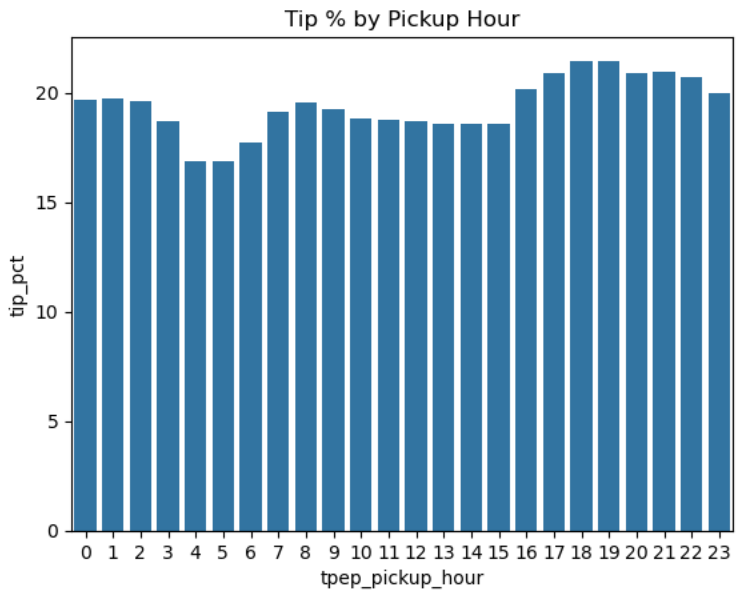
- Solo passengers tip percentage are relatively high.
- For passenger count 4, noticeable dip in trip percentage. This indicate may be due to share rides or may be the group perceives the fare as already sufficient compensation.

- In larger groups with 5-6 passengers pay high tips maybe they feel due to number of passengers or maybe they are coming back or going to social outings.

Tip percentage based on Pickup hour:

Methodology:

- Tip percentage was calculated relative to fare amount and then remove extreme tip percentage cases for our analysis.
- Trips were grouped by tpep_pickup_hour.
- Average tip percentage was computed for each pickup hour.
- Visualization: Bar plots were used to highlight differences across average hourly tip percentage.



Analysis and Insights:

- Tip percentage are lowest during early morning hours (5 AM – 7AM)
- Tip percentage in afternoon remains consistent.
- Tip percentages rise significantly, peaking around 18–19 hrs. This period coincides with leisure activities, social outings, and return commutes.
- Drivers working evening shifts benefit from higher tip percentages, while morning shifts yield lower relative tips.
- As operational strategy Operators need to provide incentives for morning shift drivers to offset lower tipping behavior.

- Understanding tipping behavior by time of day helps assess driver compensation fairness.

What factors lead to low tip percentages?

- Passengers generally perceive a fixed tip amount for long trips, lowering the percentage.
- Some passengers (mostly travelers) consistently tip low, influenced by local tipping practice.
- Tipping declines during share rides, when group dynamics make responsibility unclear.
- Travel purpose and passenger mood, for example early morning rides where passenger wants to catch a flight or train prioritizing efficiency. People tend to pay more tips during evening reflecting leisure travel and social outings.

Compare trips with tip percentage < 10% to trips with tip percentage > 25%

Methodology:

- High tip percentage outliers were removed from the dataset, as they could adversely impact the analysis if included.
- Trips were divided into two groups: Low tips (<10%) and high tips (>25%)
- Average fare, trip distance and passenger count were compared across groups.

Analysis and Insights:

- Low tip trips often associated with longer trips, higher fare or early morning travel.
- Group travel shows lower tipping due to unclear responsibility.
- High tipping behavior is common in short trips.
- Solo passenger and small groups tend to tip higher.

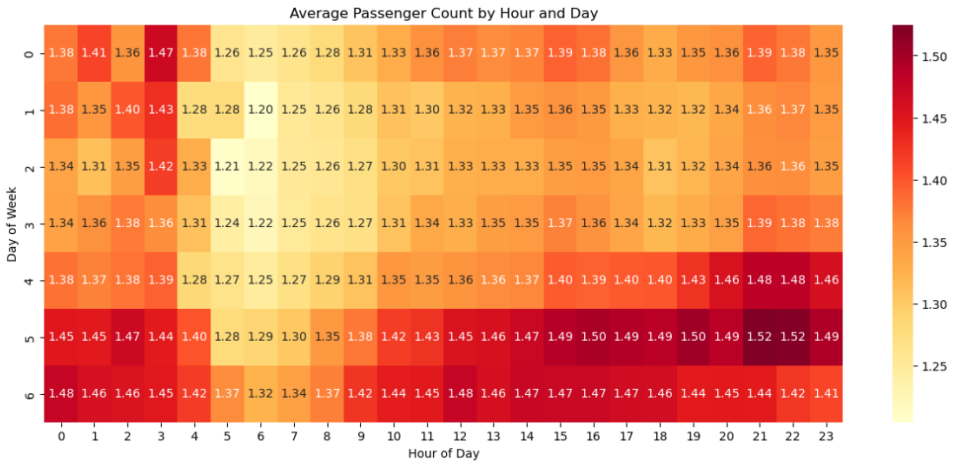
3.2.14. Analyse the trends in passenger count

Methodology:

- Trips were grouped by weekday and tpep_pickup_hour.
- Average passenger count was computed for each group.
- Pivot data created for getting better insight. Where rows are defined as weekday and columns are defined as hour.

```
pivot_data = passenger_summary.pivot(
    index="weekday",
    columns="tpep_pickup_hour",
    values="passenger_count"
)
```

- Visualization: Heat map was used to highlight differences across weekday and hour.
- Lighter shade in heat map indicates solo trip with less average passenger count.
- Darker shade indicates more group trips.



Analysis and Insights:

- Average passenger count remains close to 1.3 -1.5 throughout which indicates majority of rides are solo trips. Which we can verify from below query
  df_non_zero["passenger_count"].value_counts()

```
[659]: passenger_count
       1    1416361
       2     270134
       3      67114
       4      36842
       5      23535
       6      15623
```

- On weekdays passenger counts mostly ranges 1.25-1.40 dominated by office commuters.
- On weekends darker shade represent group trips increases, ore social and leisure trips.

- Weekends show consistently higher passenger counts than weekdays. Larger vehicles are more useful during weekends.

3.2.15. Analyse the variation of passenger counts across zones

Methodology**:**

- Grouping: By aggregating trips based on zones and passenger counts, we can understand how group travel patterns differ across various locations.

```
zone_pass_dist = (
    df_non_zero
    .groupby(["pu_zone","passenger_count"])
    .size()
    .reset_index(name="trip_count")
)
```

- Normalization: Converting the values into percentages makes it easier to compare zones fairly.

```
zone_pass_dist["pct"] = (
    zone_pass_dist
    .groupby("pu_zone")["trip_count"]
    .transform(lambda x: x / x.sum() * 100)

)
```

- Sort the values based on percentage and trip count and then group by passenger count to find maximum percentage with maximum number of trips for each passenger count.
  For example, if too many records are with maximum percentage 100 for passenger count 1. We want to pick the record with maximum number of trips with percentage 100.

```
zone_pass_dist=zone_pass_dist.sort_values(["pct","trip_count"],ascending=False)
max_zone_per_passenger =(zone_pass_dist.loc[ zone_pass_dist.groupby("passenger_count")["pct"].idxmax()].reset_index(drop=True))

print(max_zone_per_passenger)
```

```
                          pu_zone  passenger_count  trip_count         pct
0                       Co-Op City                1         142  100.000000
1                     Crotona Park                2           1   50.000000
2                     Country Club                3           1   33.333333
3                         Red Hook                4          18    8.866995
4  Saint Michaels Cemetery/Woodside               5           2    9.523810
5               Ocean Parkway South               6           1    9.090909
```

Analysis and Insights:

- The majority of taxi trips are single-passenger rides, reflecting commuting and individual travel behavior.

- Taxi operators can focus on deploying standard vehicles in areas where solo travel is more common, while placing larger vehicles in zones that frequently experience higher group travel. This helps improve service efficiency and better match vehicle availability with passenger demand.

3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

Methodology:

- Trip records where extra > 0 are put into a new DataFrame (extra_charges_trips_df).
- Trips are grouped by pick up zone ,drop off zones and count of trips are calculated. Sorting zones with the highest frequency of extra charges.
  pickupZonesWithExtraCharges = extra_charges_trips_df.groupby(["pu_zone"])["extra"].size().reset_index(name="extraCharges_trip_count").sort_values("extraCharges_trip_count",ascending =False)
  print(pickupZonesWithExtraCharges.head())
- Trips are grouped by pickup hour (tpep_pickup_hour) and weekday. This reveals time-of-day and day-of-week patterns.

Analysis and Insights:

- LaGuardia Airport (62,000 trips) show the highest extra charges frequency in pickup zones.
- Upper East Side North (47,325 trips) show the highest extra charges frequency in drop off zones.
- Peak surcharge hours are 17:00–21:00 (5–9 PM), with counts exceeding 90,000 trips per hour.
- Midweek (Thursday) shows the highest surcharge count.
- Maintain a fleet of larger vehicles at airports to maximize revenue from surcharge-heavy trips.
- We can use surcharge frequency data to forecast high revenue periods and zones and manage fleet in those zones accordingly.

4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

- Zones with consistently high pickup volumes should be given priority in vehicle deployment. This reduces passenger wait times and improves driver utilization.
- Since most of the trips involve one or two passengers, standard vehicles should remain the primary fleet type. However, larger vehicles should be allocated to zones and time periods where group travel is more frequent.
- Driver shift should be aligned with demand patterns. More drivers should be scheduled during demand period. Incentives can be offered to encourage availability.
- Routing algorithms should integrate real-time traffic conditions to avoid congestion-prone routes. This helps reduce trip duration, fuel consumption, and passenger dissatisfaction.
- Zones with high pickup/drop off ratio indicate vehicle shortage from our analysis. It can be further used to rebalance vehicle across zones.
- By following above recommendations we can reduce passenger wait time, increase driver productivity and lower operational cost.

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analyzing trip trends across time, days and months.

- Demand varies significantly throughout the day. During Weekday mornings and evenings, business and residential zones experience higher pickups. Therefore, higher number of cabs should be positioned in those areas.
- Operators should deploy sedan in these areas, as percentage of solo travelers are more.
- Reduce late night allocations in office areas due to less demand.
- Maintain larger vehicles fleet near entertainment areas on weekend as group travels are more frequent on weekend in comparison to weekdays.
- Apply surcharge pricing to balance demand and incentivize drivers.

- On weekdays focus on office areas to maximize the profit.
- On weekends target residential and entertainment zones.
- Similarly, as per month analysis, in summer focus on larger vehicles as group trips are more often in comparison to other months.

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

- Analysis shows that demand peaks during weekday rush hours, weekend evenings. During these high-demand windows, moderate surge pricing can be applied to increase revenue. lower fares during off-peak hours can help stimulate demand and improve vehicle utilization.
- It might be possible that people in zones with a smaller number of trips use public transports more often due to cost effectiveness. Provide offers to those passengers to increase number of trips in those zones.
- Analysis shows that people tend to tip generously on shorter trips as well. Provide proper training to drivers, maintaining hygiene in the cabs can improve customer satisfaction and in return increase the income of drivers as well.
- During time, day of the week and months with low demands, provide promotions, discounts to increase revenue.
- Our vendor rates analysis at different hours of the day clearly indicates the difference between vendors pricing. Excessive fare may lead to customer churn, hence strategize it as per market price.
- Loyalty programs can be introduced to incentivize loyal customers and to retain the existing customers.